

VRChat Paper Tablet UI Systeem

Producthandleiding, architectuurdokument en AI-startprompt

Versie 1.5 - 2026-05-21

Voor StefanieInVR werelden - Open Classroom, cinema, festival, kunstwereld en toekomstige VRChat projecten

Versie	Datum	Status	Belangrijkste wijziging
1.0	2026-05-17	Basisdocument	Eerste producthandleiding en architectuur.
1.1	2026-05-19	VIP addendum	VIP access, tablet lock, stage blocker en lokale filtering.
1.2	2026-05-20	Geteste versie	Managers op Default layer, ownership-safe sync, visitor guide PowerPoint/MP4.
1.3	2026-05-21	Vorige mijlpaal	ScreenReadabilityManager : global brightness/contrast, local table/local player sliders, global override, global/local reset en late join getest.
1.4	2026-05-21	Huidige mijlpaal	GlobeTransformManager toegevoegd: globe pickup, global yaw, global scale, losse UI-handle met VRC Parent Constraint, reset via object-reset plus manager-reset, in VR getest.
1.5	2026-05-21	Geteste uitbreiding	Hybrid Object Toggle systeem: global/local override voor objecten, getest met Bird en sprite-toggle visuals.

Actuele status: Het globe systeem is in VR getest en werkt goed. Daarna is het Hybrid Object Toggle systeem v1.5 toegevoegd en getest met de Bird: global aan/uit, local override, sprite-toggle visuals en global override werken. Vanaf deze mijlpaal eerst projectbackup maken voordat nieuwe uitbreidingen worden toegevoegd.

Inhoud

1. Wat dit systeem is
2. Waarom het is gemaakt
3. De hoofdregel
4. Huidige hierarchy en prefab-denken
5. Netwerk- en late-join architectuur
6. Managers en scripts
7. Screen Readability module v1.3
8. Globe module v1.4
9. Reset systeem
10. VIP access, tablet lock en stage blocker

StefanieInVR - Paper Tablet UI Systeem v1.5 - Manager = waarheid | Button = input | Visual = lokale weergave

11. Visitor guide, playlists en classroom tools
12. Testplan en huidige teststatus
13. Debuggen met de Nova Scene Tablet Analyzer
14. Veilig uitbreiden naar andere werelden
15. Veelvoorkomende fouten
16. AI-startprompt voor toekomstige chats
17. Script-inventaris
18. Toekomstige uitbreidingen
19. Hybrid Object Toggle module v1.5

1. Wat dit systeem is

De VRChat Paper Tablet UI is een herbruikbaar tablet-/clipboard-systeem voor VRChat worlds. Visueel voelt het als een papieren tablet of clipboard met tabs en world-space UI. Technisch bestaat het uit UdonSharp managers, knoppen, sliders, access filtering, lokale visuals en losse modules zoals screen readability en globe controls.

Het systeem is gebouwd voor de Open Classroom, maar moet als product/prefab-achtig systeem bruikbaar blijven in andere StefaniInVR werelden, zoals een cinema, festivalwereld, kunstwereld, showcase world of eventruimte.

- Gebruikers bedienen publieke functies via duidelijke tabs.
- Hosts en VIPs krijgen extra beheerfuncties achter lokale access filtering.
- Late joiners krijgen de juiste status via manager-state.
- Bezoekers kunnen lokaal hun schermleesbaarheid aanpassen zonder anderen te storen.
- De globe kan nu fysiek worden opgepakt, globaal worden gedraaid, globaal worden vergroot en met een losse UI-handle worden bediend.
- AI kan later sneller begrijpen waar de waarheid staat en waar alleen visuals/input zitten.

2. Waarom het is gemaakt

Standaard Unity UI in VRChat wordt snel rommelig als elke knop of slider zelf status bewaart. Dat veroorzaakt problemen met multiplayer, late join, admin-only content, panels die uit staan, gedeelde materials en Quest-performance. Dit systeem voorkomt dat door de echte status op altijd actieve manager-objecten te zetten.

- Een duidelijke, vriendelijke tablet voor bezoekers.
- Een robuuste structuur voor multiplayer en late join.
- Alleen compacte data syncen: ints, bools, floats en bitmasks op managers.
- Geen UI-objecten, sprites, hover states, renderers of material objects over het netwerk syncen.
- Material floats en visuals worden lokaal toegepast op elke client.
- Een systeem dat later documenteerbaar, verkoopbaar en uitbreidbaar blijft.

3. De hoofdregel

Hoofdregel: Manager = waarheid. Button = input. Visual = lokale weergave.

Onderdeel	Rol	Mag syncen?
Manager	Bewaart echte status en stuurt visuals of appliers aan.	Ja, als de status globaal moet zijn.
Button / input link	Geeft klik, tabkeuze, slideractie of reset door aan een manager.	Nee.
Visual / applicator	Toont lokaal wat de manager zegt: sprites, active states, material floats, transform scale of child-rotation.	Nee.
Panel / canvas	Alleen UI-weergave. Mag aan/uit of via constraint volgen.	Nee.
Pickup root	Fysiek object dat door VRChat wordt opgepakt en met VRCObjectSync beweegt.	Transform sync via VRCObjectSync, geen slider-waarheid.

Gevolg: een knop of slider mag nooit de netwerkwaarheid zijn. Een knop of slider roept alleen een managerfunctie aan. De manager bewaart de state en zorgt dat late joiners die state kunnen ontvangen.

4. Huidige hierarchy en prefab-denken

De belangrijke runtime managers staan los onder UIs/Managers. Dit is bewust: managers mogen niet onder disabled panels, VIP-content, pickup objects of geschaalde visual roots staan. Panels en visual roots mogen wel aan en uit.

Classroom

- | - UIs
 - | | - Sync Video Player
 - | | - EventSystem
 - | | - Managers
 - | | | - VipAccessManager
 - | | | - TXL Screen Auto Visibility
 - | | | - Projector Screen Toggle
 - | | | - Tab Manager
 - | | | - Toggle Manager
 - | | | - Reset Objects Manager
 - | | | - ScreenReadabilityManager
 - | | | - GlobeTransformManager
- | |
- | | - Other Toggles and Systems
 - | | | - Globe System
 - | | | | - Globe_UI_Handle_Home
 - | | | | - Globe_UI_Handle
 - | | | | - GameObject_Home
 - | | | | - Globe_toy
 - | | | | | - GlobeScaleRoot
 - | | | | | - Globe_Pivot
 - | | | | | - Globe
 - | | | | - Canvas_Globe_Holder
 - | | | | | - Paper_Note_Canvas_Globe
 - | | | | | | - Canvas (Globe)
 - | | | | | | - Slider (Rotate Globe)
 - | | | | | | - Slider (Scale Globe)
 - | |
 - | | - Paper Tablet System
 - | | - Table System
 - | | - Public Player Controls

- UIs/Managers en alle manager-objecten daaronder moeten altijd actief blijven.
- Managers staan op layer Default, niet op Pickup.
- Alleen echte pickup-objecten horen op Pickup of hebben pickup/collider gedrag.
- GlobeTransformManager staat onder UIs/Managers en blijft altijd actief.
- Globe_toy is de fysieke pickup-root. GlobeScaleRoot is alleen een visual scale root.
- Canvas_Globe_Holder of de globe UI canvas volgt de Globe_UI_Handle via VRC Parent Constraint.

5. Netwerk- en late-join architectuur

Het systeem syncet alleen kleine managerwaarden. Visuals worden lokaal opnieuw opgebouwd op basis van die waarden. Daardoor blijft het licht en Quest-vriendelijk.

Manager	Gesyncte waarde	Type	Waarom klein?
PaperTabletTabManager	selectedTabIndex	int	Alleen het tabnummer wordt gesynct.
VipAccessManager	refreshVersion, tabletLockEnabled, stageBlockerEnabled	int / bool / bool	VIP status zelf blijft lokaal; alleen refresh en globale locks syncen.
PaperTabletToggleManager	stateBits	int bitmask	Maximaal 32 global toggles in 1 getal. Local toggles blijven lokaal.
PaperTabletSliderManager	value0..value7	int 0..1000	Legacy/general slider-eindwaarden. Niet voor screen readability en niet

StefaniInVR - Paper Tablet UI Systeem v1.5 - Manager = waarheid | Button = input | Visual = lokale weergave

			voor globe v1.4.
ScreenReadabilityManager	globalBrightness, globalContrast	float / float	Alleen globale standaardwaarden syncen. Material floats worden lokaal toegepast.
GlobeTransformManager	syncedYaw01, syncedScale01	float / float	Alleen slider-normalized waarden syncen. Rotatie en scale worden lokaal op child transforms toegepast.
ProjectorScreenToggle	isOpen	bool	Alleen open/dicht status.
ResettableObject	geen compacte manager-state	object sync indien nodig	Fysieke objectreset gebruikt VRObjectSync/ownership .
HybridObjectToggleManager	globalStateBits, globalRevision, hasInitializedNetworkState	int bitmask / int / bool	Global object states worden compact gesyncd. Local overrides blijven client-only.

- Niet syncen: hover state, pressed state, sprites, tekstkleur, layout, UI transforms, slider UI-objecten, renderers en lokale material-applies.
- OnDeserialization moet visuals opnieuw refreshen.
- RequestSerialization hoort pas na ownership. Voor belangrijke managers wordt ownership-safe sync gebruikt.
- Voor global sliders wordt delayed sync gebruikt, zodat slepen geen netwerkspam veroorzaakt.
- Pickup position/rotation en slider scale/yaw zijn gescheiden verantwoordelijkheden.

6. Managers en scripts

Script	Sync mode	Synced velden	Rol
PaperTabletTabManagers	Manual	selectedTabIndex	Manager voor geselecteerde tab, panels, tabstates en VIP-filter aanroep.
VipAccessManager.cs	Manual	tabletLockEnabled, stageBlockerEnabled, refreshVersion	VIP-lijst, tablet lock, stage blocker en lokale access filtering.
PaperTabletToggleManager.cs	Manual	stateBits	Global/local toggles via bitmask en lokale states.
PaperTabletSliderManager.cs	Manual	value0..value7	Legacy/general slider manager. Niet meer de screen readability manager en niet de globe manager.
ScreenReadabilityManager.cs	Manual	globalBrightness, globalContrast	Global/local brightness en contrast met local override en global reset.
ScreenReadabilityInput.cs	None	-	Input bridge voor sliders en reset buttons.
GlobeTransformManager.cs	Manual	syncedYaw01, syncedScale01	Nieuwe v1.4 manager voor globe yaw, globe scale, reset van globe controls en late join refresh.
ProjectorScreenToggle.cs	Manual	isOpen	Projector open/dicht, blendshape en button visuals.
TableScreenManager.cs	None	-	Lokale table screen open/close en scale slider.
ResetObjectManager.cs	Unspecified	-	Centrale resetmanager voor groepen of alles.
ResettableObject.cs	Unspecified	-	Component op objecten die

			naar hun home transform kunnen resetten.
TXLScreenAutoVisibility.cs	None	-	Lokaal hulpscript voor TXL/video/projector visibility.
HybridObjectToggleManager.cs	Manual	globalStateBits, globalRevision, hasInitializedNetworkState	Manager voor global/local object toggles. Global state syncet, local override blijft lokaal.
HybridObjectToggleInputs	None	-	Inputscript voor global of local knop. Roept alleen managerfuncties aan.
HybridObjectToggleGroup.cs	None	-	Objectgroep voor targets en sprite-toggle visuals. Past effectieve state lokaal toe.

7. Screen Readability module v1.3

Brightness en contrast zijn sinds v1.3 uit het oude algemene slider-applier systeem gehaald en ondergebracht in ScreenReadabilityManager. Dit blijft ongewijzigd in v1.4.

Actie	Effect lokaal	Effect bij anderen	Sync?
Global brightness slider	Past brightness direct lokaal toe en zet lokale waarde gelijk aan global.	Na delay krijgen andere clients dezelfde global brightness en worden hun local waarden overruled.	Ja, delayed sync.
Global contrast slider	Past contrast direct lokaal toe en zet lokale waarde gelijk aan global.	Na delay krijgen andere clients dezelfde global contrast en worden hun local waarden overruled.	Ja, delayed sync.
Global reset	Zet brightness en contrast terug naar default.	Alle clients worden teruggezet naar default; local sliders springen mee.	Ja, direct sync.
Local brightness/contrast	Past alleen deze client toe.	Geen effect.	Nee.
Late join	Krijgt globalBrightness/globalContrast en bouwt lokaal materialwaarden op.	-	Via manager-state.

Belangrijke status: PaperTabletSliderManager blijft bestaan voor legacy/general sliders, maar is niet meer de waarheid voor screen brightness/contrast.

8. Globe module v1.4

8.1 Doel

De globe module maakt van de wereldbol een volledig bruikbaar classroom-object. De globe kan fysiek worden opgepakt/verplaatst, de UI kan los via een handle worden opgepakt, en de globe kan globaal worden gedraaid en vergroot met sliders.

- Iedereen ziet dezelfde globe yaw/rotatie.
- Iedereen ziet dezelfde globe schaal.
- De globe start op normale schaal 1 en kan alleen groter worden.
- De rotate slider start op 0 graden via Default Yaw 01 = 0.5.
- De scale slider start links op Value 0, wat gelijk is aan schaal 1.

StefanieInVR - Paper Tablet UI Systeem v1.5 - Manager = waarheid | Button = input | Visual = lokale weergave

- Pickup root en scale root blijven gescheiden zodat VRCObjectSync niet op een geschaald object staat.
- De UI-handle kan los worden opgepakt via VRC_Pickup en VRCObjectSync.
- De UI canvas volgt de handle via VRC Parent Constraint.

8.2 Structuur en verantwoordelijkheden

Object	Rol	Belangrijke componenten / instellingen
GlobeTransformManager	Manager en netwerkwaarheid voor yaw en scale.	UdonSharp, Manual sync, staat onder UIs/Managers, layer Default.
Globe_toy	Fysieke globe pickup-root.	VRC_Pickup, Rigidbody, Collider, VRCObjectSync, ResettableObject. Local scale blijft 1,1,1.
GlobeScaleRoot	Visual root die wordt geschaald door scale slider.	Geen VRC_Pickup en geen VRCObjectSync. Alleen visual scale.
Globe_Pivot	Yaw pivot die wordt gedraaid door rotate slider.	Child van GlobeScaleRoot. Draait lokaal rond localYawAxis.
Globe	Mesh/visual van de wereldbol.	Mesh Renderer / material. Geen netwerkwaarheid.
Globe_UI_Handle	Losse pickup-handle voor de globe UI.	VRC_Pickup, Rigidbody, Collider, VRCObjectSync.
Canvas_Globe_Holder / Canvas (Globe)	World-space UI met rotate en scale slider.	Volgt Globe_UI_Handle via VRC Parent Constraint. Bewaart geen netwerkwaarheid.

8.3 GlobeTransformManager inspector

Veld	Waarde	Waarom
Pickup Root	Globe_toy	Referentie naar fysieke root. Wordt niet geschaald door de manager.
Scale Root	GlobeScaleRoot	Alleen deze child wordt geschaald.
Yaw Pivot	Globe_Pivot	Alleen deze child wordt gedraaid.
Rotate Slider	Slider (Rotate Globe)	Input voor OnRotateSliderChanged.
Scale Slider	Slider (Scale Globe)	Input voor OnScaleSliderChanged.
Min Yaw	-180	Sliderwaarde 0 wordt -180 graden.
Max Yaw	180	Sliderwaarde 1 wordt 180 graden.
Default Yaw 01	0.5	Midden tussen -180 en 180 is 0 graden.
Min Scale	1	Normale startgrootte. De globe wordt niet kleiner dan normaal.
Max Scale	5	Maximale vergroting. Kan later lager/hoger worden getuned.
Default Scale 01	0	Slider links is normale schaal 1.
Synced	aan	Yaw en scale moeten globaal zichtbaar zijn.
Sync Delay	0.35	Delayed sync voorkomt netwerkspam tijdens slepen.

8.4 Slider instellingen

Slider	Min	Max	Start Value	Event
Slider (Rotate Globe)	0	1	0.5	OnValueChanged -> GlobeTransformManager -> SendCustomEvent("OnRotateSliderChanged")
Slider (Scale Globe)	0	1	0	OnValueChanged -> GlobeTransformManager -> SendCustomEvent("OnScaleSliderChanged")

				OnScaleSliderChanged")
--	--	--	--	------------------------

8.5 UI handle met VRC Parent Constraint

De globe UI is losgemaakt van de globe zelf. De UI wordt niet opgepakt als deel van Globe_toy, maar volgt een aparte UI-handle. Dit maakt de bediening flexibeler in VR.

Globe System

| - Globe_UI_Handle

| | - VRC_Pickup

| | - Rigidbody

| | - Collider

| | - VRCObjectSync

|

| - Globe_toy

| | - GlobeScaleRoot

| | | - Globe_Pivot

| | | - Globe

|

| - Canvas_Globe_Holder

| | - VRC Parent Constraint

| | - Paper_Note_Canvas_Globe

| | | - Canvas (Globe)

| | | | - Slider (Rotate Globe)

| | | | - Slider (Scale Globe)

- Constraint staat op Canvas_Globe_Holder of op het canvas-object dat de hele UI moet laten volgen.
- Source = Globe_UI_Handle.
- Weight = 1.
- Target Transform mag leeg blijven als de constraint op het te volgen object staat.
- Eerst handle en UI mooi positioneren, daarna Activate klikken.
- De UI blijft input; de manager blijft waarheid.

8.6 Resetgedrag van de globe

De globe gebruikt twee soorten reset, omdat positie/rotatie en sliderwaarden op verschillende plekken wonen.

Reset onderdeel	Wat reset het?	Waar geregeld?
Fysieke reset	Globe_toy position en rotation terug naar GameObject_Home.	ResettableObject op Globe_toy via ResetObjectManager.
Slider/control reset	Yaw terug naar default, scale terug naar normaal, sliders refreshen, state syncen.	GlobeTransformManager.ResetGlobeControls.
UI handle reset	Optioneel: Globe_UI_Handle terug naar Globe_UI_Handle_Home.	ResettableObject op Globe_UI_Handle indien gewenst.

Belangrijk: op Globe_toy moet Reset Scale uit blijven. Globe_toy is de pickup/objectsync root en moet local scale 1,1,1 houden. De schaal van de zichtbare globe wordt via GlobeScaleRoot geregeld.

Een globale resetknop voor de globe hoort daarom twee acties te hebben:

1. ResetObjectManager.ResetGroupX of ResetObjectManager.ResetEverything voor Globe_toy position/rotation.
2. GlobeTransformManager.SendCustomEvent("ResetGlobeControls") voor yaw/scale/sliders.

8.7 Waarom scale niet op VRCObjectSync root zit

De scale slider schaaft bewust niet Globe_toy. Globe_toy is de pickup root met VRC_Pickup, Rigidbody, Collider en VRCObjectSync. Als die root meeschaalt, kan pickup/collider/object sync onduidelijk of instabiel worden. Daarom blijft Globe_toy altijd schaal 1,1,1 en schaaft alleen GlobeScaleRoot.

StefanieInVR - Paper Tablet UI Systeem v1.5 - Manager = waarheid | Button = input | Visual = lokale weergave

9. Reset systeem

ResetObjectManager staat onder UIs/Managers. ResettableObject staat op objecten die terug naar een home transform moeten. Voor fysieke objecten die iedereen hetzelfde moet zien, gebruikt ResettableObject ownership en VRObjectSync waar nodig.

Gebruik	Instelling	Opmerking
Gewone globale pickup reset	Reset Position aan, Reset Rotation aan, Reset Scale meestal uit, Reset For Everyone aan.	Gebruik pickupObject en objectSync references als die niet automatisch goed gevonden worden.
Globe_toy	Reset Position aan, Reset Rotation aan, Reset Scale uit.	Scale wordt niet door ResettableObject gedaan.
GlobeScaleRoot	Geen ResettableObject nodig.	Wordt door GlobeTransformManager geregeld.
Globe_UI_Handle	Optioneel ResettableObject naar Globe_UI_Handle_Home.	Handig als de UI-handle terug moet kunnen.

10. VIP access, tablet lock en stage blocker

De TabManager bepaalt welke tab globaal actief is. De VipAccessManager bepaalt lokaal wat deze speler wel of niet mag zien of bedienen. Dit blijft ongewijzigd in v1.4.

Situatie	Niet-VIP ziet	Niet-VIP mag klikken?	VIP ziet / mag
Tablet unlocked, normale tab	Gewone publieke content.	Ja, normale tabs werken.	VIP kan alles bedienen.
Tablet unlocked, VIP-tab	Alleen TabletLockedRoot. Geen VipContentRoot en geen geheime bediening.	Ja, VIP-tab mag open, maar toont access-info.	VIP ziet VipContentRoot en VIP-tools.
Tablet locked, elke tab	Altijd alleen TabletLockedRoot. Tabs zichtbaar, content uit.	Nee, tabklikken wordt genegeerd.	VIP kan tabs blijven bedienen en lock uitzetten.
Stage blocker aan	Blocker-colliders actief voor niet-VIP.	Niet relevant.	VIP loopt door; blockers lokaal uit.

Security note: dit is een praktisch VRChat access-systeem, geen harde DRM. Zet er geen geheime assets, wachtwoorden of betaalde content achter die echt beschermd moet blijven.

11. Visitor guide, playlists en classroom tools

De Open Classroom Visitor Guide blijft onderdeel van het systeem. Deze visitor guide is niet de technische manual. De visitor guide legt bezoekers simpel uit wat ze kunnen doen; deze producthandleiding legt uit hoe het systeem technisch werkt.

Onderdeel	Doel
Open Classroom Visitor Guide	Korte uitleg van tablet, tabs, video player en visitor manners.
Derafog VRChat guide	Algemene VRChat gids voor bezoekers die nog geen creator zijn.
CG Cookie Blender beginner course	Basis Blender cursus.
Iconoclass VRChat world guide	Eerste VRChat world maken.
Optimization tutorials	Optimalisatie mindset voor 3D/VR.

12. Testplan en huidige teststatus

12.1 Geteste status v1.4

- Globe systeem is in VR getest en werkt goed.
- Globe_toy kan worden opgepakt en verplaatst.
- Globe yaw slider werkt en syncet globaal.
- Globe scale slider werkt en syncet globaal.
- Globe start op schaal 1 en wordt alleen groter.
- Rotate default staat op 0 graden via Default Yaw 01 = 0.5.
- Scale default staat op normale grootte via Default Scale 01 = 0.
- UI handle kan los van de globe worden opgepakt.
- UI canvas volgt de handle via VRC Parent Constraint.
- Reset werkt met combinatie van ResettableObject en ResetGlobeControls.

12.1b Geteste status v1.5

- Hybrid Object Toggle systeem is getest met de Bird.
- Global Bird toggle zet de Bird voor iedereen aan of uit.
- Local Bird toggle wijzigt alleen de zichtbaarheid voor de lokale speler.
- Een nieuwe global actie overrulet lokale afwijkingen en trekt de local visual weer gelijk.
- Sprite-toggle visuals werken met een onSprite en offSprite op 1 Image per global/local knop.
- Bestaande PaperTabletToggleManager is niet aangepast en blijft bedoeld voor de bestaande werkende toggles.

12.2 Testchecklist na wijzigingen

- Test in editor/playmode alleen voor basisgedrag.
- Test in VRChat met minstens twee clients/accounts voor sync.
- Test late join: speler A zet yaw/scale, speler B joint later en moet dezelfde yaw/scale zien.
- Test reset: positie/rotatie terug, scale terug naar 1, rotate slider naar 0 graden, scale slider naar links.
- Test pickup: Globe_toy blijft schaal 1,1,1 en pickup blijft normaal voelen.
- Test UI handle: handle beweegt UI, sliders blijven interactable, constraint blijft goed.

13. Debuggen met de Nova Scene Tablet Analyzer

Er is een Editor script dat de volledige Unity hierarchy kan uitlezen. De werkende oude versie mag voorlopig gebruikt worden. De poging om extra checks toe te voegen gaf een foutmelding en is uitgesteld voor later.

Onderdeel	Waarde
Script	Assets/Editor/NovaSceneTabletAnalyzer.cs of huidige werkende analyzer-locatie.
Menu	Nova > Scene Tools > Analyze Current Scene
Output	TXT, Markdown, JSON en CSV in Assets/NovaSceneReports
Gebruik	Upload vooral TXT of MD naar Nova bij fouten.

Debugworkflow: draai analyzer, upload TXT/MD, laat Nova zoeken naar lege references, verkeerde slider events, dubbele managers, oude button targets, managers onder verkeerde parents, en GlobeTransformManager references.

14. Veilig uitbreiden naar andere werelden

Wereldtype	Mogelijke tab-/toolinhoud
Classroom	Video, playlists, pens, laser pointer, globe, learning tools.
Cinema / lecture hall	Screen mode, curtain control, talk mode lights, cinema lights.
Festivalwereld	Stage effects, dancefloor lights, info signs, event controls.
Kunstwereld	Guided tour, artwork info, lighting scenes, audio guide.

StefanieInVR - Paper Tablet UI Systeem v1.5 - Manager = waarheid | Button = input | Visual = lokale weergave

Showcase world	Prefab demos, credits, options, public controls.
----------------	--

- Houd de kernarchitectuur hetzelfde.
- Nieuwe globale status hoort op een altijd actieve manager onder UIs/Managers.
- Gebruik Inspector references; hardcode geen classroom-specifieke objectnamen als dat niet nodig is.
- Maak tabs, panels en tools vervangbaar per wereld.
- Test altijd local, daarna multiplayer, daarna late join.
- Documenteer nieuwe functies pas nadat ze getest zijn.

15. Veelvoorkomende fouten

Probleem	Waarschijnlijke oorzaak	Fix
Globe start te groot	Default Scale 01 staat niet op 0 of Min Scale staat onder 1.	Zet Default Scale 01 = 0, Min Scale = 1, scale slider Value = 0.
Globe draait naar verkeerde beginhoek	Default Yaw 01 staat op 0 of 1 in plaats van 0.5.	Zet Default Yaw 01 = 0.5 en rotate slider Value = 0.5.
Pickup voelt raar na schalen	Scale zit op Globe_toy of VRCObjectSync root.	Alleen GlobeScaleRoot schalen. Globe_toy scale 1,1,1 houden.
Reset zet positie terug maar scale/yaw niet	Resetknop roept alleen ResetObjectManager aan.	Voeg tweede event toe: GlobeTransformManager.ResetGlobeControls.
UI beweegt niet mee met handle	Constraint mist source of Activate is niet gebruikt.	Source = Globe_UI_Handle, Weight = 1, daarna Activate.
Canvas schaalt mee met globe	Canvas staat onder GlobeScaleRoot.	Zet canvas/holder buiten GlobeScaleRoot.
Late join ziet verkeerde globe state	Manager staat niet actief onder UIs/Managers of OnDeserialization refresh ontbreekt.	Manager actief houden, Manual sync, references controleren.
Button/slider doet niets	OnClick/OnValueChanged target of eventnaam verkeerd.	Gebruik SendCustomEvent met exacte functie: OnRotateSliderChanged, OnScaleSliderChanged, ResetGlobeControls.

16. AI-startprompt voor toekomstige chats

Ik werk aan mijn VRChat Paper Tablet UI systeem voor mijn Open Classroom world.

Gebruik de actuele v1.4 handleiding en huidige scripts als bron van waarheid. Oudere documenten v1.0, v1.1, v1.2 en v1.3 zijn historisch. v1.4 is leidend.

Techniek:

- Unity 2022.3.22f1
- VRChat SDK
- UdonSharp
- Quest-vriendelijk
- world-space UI
- managers staan onder UIs/Managers
- managers moeten altijd actief blijven
- managers staan op layer Default, niet Pickup
- buttons en visuals bewaren geen netwerkwaarheid
- hoofdregel: manager = waarheid, button = input, visual = lokale weergave

Werkende status v1.4:

StefanieInVR - Paper Tablet UI Systeem v1.5 - Manager = waarheid | Button = input | Visual = lokale weergave

- tabs werken
- toggles werken
- VIP access werkt
- tablet lock werkt
- stage blocker werkt
- projector screen toggle werkt
- reset object systeem werkt
- visitor guide MP4/playlists werken
- ScreenReadabilityManager werkt
- global brightness/contrast op hoofdtablet werkt
- local brightness/contrast op table screens werkt
- local brightness/contrast op local player controller werkt
- global reset overrulet local readability waarden
- local reset reset alleen lokale speler
- late join is getest en werkt
- GlobeTransformManager werkt
- globe kan fysiek worden opgepakt en verplaatst
- globe yaw slider werkt global/synced
- globe scale slider werkt global/synced
- globe start op schaal 1 en kan alleen groter worden
- globe UI handle kan los worden opgepakt
- globe UI volgt handle via VRC Parent Constraint
- globe reset gebruikt ResettableObject plus ResetGlobeControls
- globe systeem is in VR getest en werkt goed

Belangrijk over brightness/contrast:

Het oude systeem met PaperTabletSliderManager + PaperTabletMaterialFloatApplier voor brightness/contrast is vervangen door:

- ScreenReadabilityManager.cs
- ScreenReadabilityInput.cs

Belangrijk over globe:

- GlobeTransformManager.cs is de waarheid voor globe yaw en globe scale.
- Globe_toy is pickup root met VRC_Pickup, Rigidbody, Collider en VRCObjectSync.
- Globe_toy blijft local scale 1,1,1.
- GlobeScaleRoot is alleen visual scale root.
- Globe_Pivot is yaw pivot.
- Canvas_Globe_Holder / Canvas (Globe) volgt Globe_UI_Handle via VRC Parent Constraint.
- Reset van globe bestaat uit fysieke reset plus ResetGlobeControls.

Werkstijl:

- Verander bestaande werkende scripts zo weinig mogelijk.
- Geef bij code altijd het volledige script terug.
- Geen gokjes: als iets onzeker is, zeg dat eerlijk.
- Werk stap voor stap.
- Eerst uitleggen waarom, dan kleine praktische stappen.
- Geen grote herstructurering tenzij echt nodig.

Mogelijke volgende module:

Een herbruikbare HybridObjectToggleManager voor objecten zoals birds: global aan/uit plus lokale speler override, waarbij global wijzigingen lokale overrides kunnen overrulet.

17. Script-inventaris

Deze lijst is gebaseerd op de huidige Scripts.zip die bij v1.4 is aangeleverd. De Unity projectbestanden blijven de echte bron van de code.

Bestand	Class	Sync mode	UdonSynced velden
GlobeTransformManager.cs	GlobeTransformManager	Manual	syncedYaw01, syncedScale01
PaperTabletTabButton.cs	PaperTabletTabButton	Unspecified	-
PaperTabletTabManager.cs	PaperTabletTabManager	Manual	selectedTabIndex
PaperTabletTabState.cs	PaperTabletTabState	None	-
PaperTabletToggleButton.cs	PaperTabletToggleButton	Unspecified	-
PaperTabletToggleManager.cs	PaperTabletToggleManager	Manual	stateBits
PaperTabletToggleVisual.cs	PaperTabletToggleVisual	Unspecified	-
ProjectorScreenToggle.cs	ProjectorScreenToggle	Manual	isOpen
ResetObjectManager.cs	ResetObjectManager	Unspecified	-
ResettableObject.cs	ResettableObject	Unspecified	-
ScreenReadabilityInput.cs	ScreenReadabilityInput	None	-
ScreenReadabilityManager.cs	ScreenReadabilityManager	Manual	globalBrightness, globalContrast
TXLScreenAutoVisibility.cs	TXLScreenAutoVisibility	None	-
VipAccessManager.cs	VipAccessManager	Manual	tabletLockEnabled, stageBlockerEnabled, refreshVersion
HybridObjectToggleManager.cs	HybridObjectToggleManager	Manual	globalStateBits, globalRevision, hasInitializedNetworkState
HybridObjectToggleInput.cs	HybridObjectToggleInput	None	-
HybridObjectToggleGroup.cs	HybridObjectToggleGroup	None	-

18. Toekomstige uitbreidingen

Idee	Status	Waarom later?
HybridObjectToggleManager	Voltooid v1.5	Gebouwd als apart systeem; eerste testcase Bird is getest.
Bird local/global override	Voltooid v1.5	Bird gebruikt global/local override met sprite-toggle visuals en global override gedrag.
NovaSceneTabletAnalyzer extra issue checks	Uitgesteld.	De upgrade gaf een foutmelding. Oude analyzer voorlopig blijven gebruiken.
Globe polish	Optioneel.	Eventueel betere labels, resetknop voor UI handle, range tuning voor maxScale.

Appendix A - GlobeTransformManager.cs

Dit is de v1.4 kernmodule voor de globe. Bij toekomstige wijzigingen altijd het volledige script teruggeven en eerst testen in VRChat.

StefanieInVR - Paper Tablet UI Systeem v1.5 - Manager = waarheid | Button = input | Visual = lokale weergave

```

using UdonSharp;
using UnityEngine;
using UnityEngine.UI;
using VRC.SDKBase;

[UdonBehaviourSyncMode(BehaviourSyncMode.Manual)]
public class GlobeTransformManager : UdonSharpBehaviour
{
    [Header("Targets")]
    [Tooltip("The root pickup object. This usually has VRC_Pickup, Rigidbody, Collider and VRCObjectSync.")]
    public Transform pickupRoot;

    [Tooltip("The child object that will be scaled by the scale slider.")]
    public Transform scaleRoot;

    [Tooltip("The child pivot that will be rotated by the rotate slider.")]
    public Transform yawPivot;

    [Header("UI Sliders")]
    public Slider rotateSlider;
    public Slider scaleSlider;

    [Header("Yaw Settings")]
    public float minYaw = -180f;
    public float maxYaw = 180f;
    public Vector3 localYawAxis = Vector3.up;

    [Header("Scale Settings")]
    [Tooltip("Normal start scale. For your globe this should usually be 1.")]
    public float minScale = 1f;

    [Tooltip("Maximum enlarged scale.")]
    public float maxScale = 5f;

    [Header("Default Slider Values")]
    [Tooltip("0.5 means the rotate slider starts in the middle.")]
    [Range(0f, 1f)]
    public float defaultYaw01 = 0.5f;

    [Tooltip("0 means normal size when Min Scale is 1.")]
    [Range(0f, 1f)]
    public float defaultScale01 = 0f;

    [Header("Sync Settings")]
    public bool synced = true;

    [Tooltip("Delay after slider movement before sending network sync. Keeps it Quest-friendly.")]
    public float syncDelay = 0.35f;

    [UdonSynced] private float syncedYaw01 = 0.5f;
    [UdonSynced] private float syncedScale01 = 0f;

    private bool _ignoreSliderCallbacks;
    private bool _syncDirty;
    private float _nextSyncTime;

    private Quaternion _baseYawLocalRotation;

```

```

private Vector3 _baseScaleRootLocalScale;

private void Start()
{
    if (yawPivot != null)
    {
        _baseYawLocalRotation = yawPivot.localRotation;
    }

    if (scaleRoot != null)
    {
        _baseScaleRootLocalScale = scaleRoot.localScale;
    }

    ApplyAll(true);
}

private void Update()
{
    if (!synced) return;
    if (!_syncDirty) return;

    if (Time.time >= _nextSyncTime)
    {
        CommitSyncedValues();
    }
}

public void OnRotateSliderChanged()
{
    if (_ignoreSliderCallbacks) return;
    if (rotateSlider == null) return;

    float value = Mathf.Clamp01(rotateSlider.value);

    ApplyYaw(value, false);

    if (synced)
    {
        EnsureOwnership();
        syncedYaw01 = value;
        MarkDirtyForDelayedSync();
    }
}

public void OnScaleSliderChanged()
{
    if (_ignoreSliderCallbacks) return;
    if (scaleSlider == null) return;

    float value = Mathf.Clamp01(scaleSlider.value);

    ApplyScale(value, false);

    if (synced)
    {
        EnsureOwnership();
        syncedScale01 = value;
    }
}

```

```

        MarkDirtyForDelayedSync();
    }
}

public void ResetGlobeControls()
{
    EnsureOwnership();

    syncedYaw01 = Mathf.Clamp01(defaultYaw01);
    syncedScale01 = Mathf.Clamp01(defaultScale01);

    _syncDirty = false;

    ApplyAll(true);

    if (synced)
    {
        RequestSerialization();
    }
}

public void RefreshFromManager()
{
    ApplyAll(true);
}

public override void OnDeserialization()
{
    if (!synced) return;

    ApplyAll(true);
}

public override void OnOwnershipTransferred(VRCPlayerApi player)
{
    ApplyAll(true);
}

private void ApplyAll(bool updateSliderUI)
{
    ApplyYaw(syncedYaw01, updateSliderUI);
    ApplyScale(syncedScale01, updateSliderUI);
}

private void ApplyYaw(float value01, bool updateSliderUI)
{
    value01 = Mathf.Clamp01(value01);

    if (yawPivot != null)
    {
        Vector3 axis = localYawAxis;

        if (axis == Vector3.zero)
        {
            axis = Vector3.up;
        }
    }
}

```



```

axis.Normalize();

float angle = Mathf.Lerp(minYaw, maxYaw, value01);
Quaternion rotationOffset = Quaternion.AngleAxis(angle, axis);

yawPivot.localRotation = _baseYawLocalRotation * rotationOffset;
}

if (updateSliderUI && rotateSlider != null)
{
    _ignoreSliderCallbacks = true;
    rotateSlider.value = value01;
    _ignoreSliderCallbacks = false;
}
}

private void ApplyScale(float value01, bool updateSliderUI)
{
    value01 = Mathf.Clamp01(value01);

    if (scaleRoot != null)
    {
        float scaleValue = Mathf.Lerp(minScale, maxScale, value01);
        scaleRoot.localScale = _baseScaleRootLocalScale * scaleValue;
    }

    if (updateSliderUI && scaleSlider != null)
    {
        _ignoreSliderCallbacks = true;
        scaleSlider.value = value01;
        _ignoreSliderCallbacks = false;
    }
}

private void MarkDirtyForDelayedSync()
{
    _syncDirty = true;
    _nextSyncTime = Time.time + syncDelay;
}

private void CommitSyncedValues()
{
    if (!syncd) return;

    EnsureOwnership();

    _syncDirty = false;
    RequestSerialization();
}

private void EnsureOwnership()
{
    if (!syncd) return;
    if (Networking.LocalPlayer == null) return;

    if (!Networking.IsOwner(gameObject))
    {
        Networking.SetOwner(Networking.LocalPlayer, gameObject);
    }
}

```

```
}  
}  
}
```

19. Hybrid Object Toggle module v1.5

19.1 Doel

De Hybrid Object Toggle module is toegevoegd voor objecten die zowel globaal als lokaal bediend moeten kunnen worden. De eerste geteste toepassing is de Bird in de Open Classroom.

- Global toggle: verandert de gedeelde global state voor iedereen.
- Local toggle: verandert alleen de effectieve state voor de lokale speler.
- Een nieuwe global actie wist of negeert lokale overrides, zodat iedereen weer de global waarheid volgt.
- Late joiners krijgen de actuele global state via de manager.
- De bestaande PaperTabletToggleManager blijft ongemoeid.

19.2 Waarom dit een apart systeem is

Dit gedrag is niet hetzelfde als een gewone toggle. Een gewone toggle is of global of local. Deze module combineert beide lagen voor hetzelfde object: global waarheid plus tijdelijke lokale afwijking. Daarom is het veiliger en schoner om dit los te houden van de bestaande PaperTabletToggleManager.

19.3 Hoofdregel

Onderdeel	Rol	Sync?
HybridObjectToggleManager	Bewaart global waarheid, bitmask en revision. Stuurt refresh aan.	Ja, Manual sync.
HybridObjectToggleInput	Knopscrip. Roept global of local functie aan op de manager.	Nee.
HybridObjectToggleGroup	Past object active state en sprite visuals lokaal toe.	Nee.
Bird object / visual root	Wordt lokaal aan/uit gezet op basis van effectieve state.	Nee.

19.4 Bestanden

Bestand	Doel
HybridObjectToggleManager.cs	Manager en netwerkwaarheid voor maximaal 31 objectgroepen via bitmask.
HybridObjectToggleInput.cs	Input voor global of local knop. Kan toggle, force on, force off of local override clear doen.
HybridObjectToggleGroup.cs	Groep per object. Regelt controlled objects en sprite-toggle visuals.

19.5 Bird testcase

Voor v1.5 is bewust klein getest met alleen de Bird op toggleIndex 0.

UIs

| - Managers

| | - HybridObjectToggleManager

| | - HybridObjectToggleManager.cs

| | - HybridObjectToggleGroup.cs (Bird group mag op hetzelfde GameObject)

|

| - Paper Tablet System

| | - Global Bird Toggle Button

| | - HybridObjectToggleInput.cs

|

Classroom

| - Local Bird Toggle Button

StefanieInVR - Paper Tablet UI Systeem v1.5 - Manager = waarheid | Button = input | Visual = lokale weergave

| | - HybridObjectToggleInput.cs
 |
 Bird_Root of Bird_VisibleRoot
 | - Bird visuals / animator / sound

19.6 Inspector setup - Manager

Veld	Waarde voor Bird test	Opmerking
Default Global States	Size = 1, Element 0 = true	Bird start aan.
Groups	Size = 1, Element 0 = Bird HybridObjectToggleGroup	Mag dezelfde GameObject zijn als de manager als daar ook de Group component op staat.
Debug Logs	Uit, tenzij aan het testen	Alleen tijdelijk aanzetten bij problemen.

19.7 Inspector setup - Group

Veld	Waarde	Waarom
Manager	HybridObjectToggleManager	De group vraagt hier global/effective state op.
Toggle Index	0	Bird gebruikt index 0.
Objects On When Enabled	Bird_Root of Bird_VisibleRoot	Object dat zichtbaar/actief is wanneer de effectieve state ON is.
Objects On When Disabled	Leeg laten	Alleen nodig als er een alternatief OFF-object moet verschijnen.
Global Toggle Image	Image component van de global Bird knop	Deze image krijgt onSprite/offSprite volgens global state.
Local Toggle Image	Image component van de local Bird knop	Deze image krijgt onSprite/offSprite volgens effectieve lokale state.
On Sprite	SliderOn sprite	Sprite voor ON.
Off Sprite	SliderOff sprite	Sprite voor OFF.
Local Override Indicator	Optioneel leeg	Alleen gebruiken als je een LOCAL lampje of indicator wilt tonen.

19.8 Inspector setup - Inputs

Knop	Script	Belangrijkste waarden
Global Bird button	HybridObjectToggleInput	Manager = HybridObjectToggleManager, Toggle Index = 0, Is Global Button = true
Local Bird button	HybridObjectToggleInput	Manager = HybridObjectToggleManager, Toggle Index = 0, Is Global Button = false

Bij gewone Unity UI Buttons moet de Button OnClick verwijzen naar HybridObjectToggleInput.Press(). Bij VRChat Interact gebruikt het script ook Interact().

19.9 Sprite-toggle visuals

v1.5 gebruikt bewust een simpele sprite-wissel. Er zijn dus geen losse ON/OFF visual GameObjects nodig. Per knop is er 1 Image component, en de group wisselt alleen de sprite.

State	Global toggle image toont	Local toggle image toont
Global ON, geen local override	onSprite	onSprite
Global OFF, geen local override	offSprite	offSprite

Global ON, speler zet local OFF	onSprite	offSprite
Global OFF, speler zet local ON	offSprite	onSprite
Daarna global opnieuw gebruikt	Nieuwe global sprite	Local sprite trekt weer gelijk met global

19.10 Netwerkgedrag

- Alleen de manager syncet compacte waarden.
- globalStateBits bewaart de global ON/OFF waarden als bitmask.
- globalRevision verhoogt bij elke global wijziging.
- localOverrideBits en localValueBits zijn niet gesynct en blijven per client lokaal.
- OnDeserialization roept RefreshAll aan, zodat late joiners en andere clients de juiste global state toepassen.
- Er zijn geen Update loops nodig.

19.11 Geteste status v1.5

- Editor test geslaagd: Bird reageert op global en local toggle.
- VR test geslaagd: Bird systeem werkt in VR.
- Sprite-toggle visuals werken met onSprite/offSprite.
- Global override trekt de lokale effectieve state en local sprite weer gelijk.
- Bestaande tabs, toggles en managers hoeven hiervoor niet aangepast te worden.

19.12 Waarschuwing bij AI/NavMesh/Bird logic

Voor de eerste test is het prima om Bird_Root aan/uit te zetten. Als de Bird later NavMesh, AI, Animator state, audio of random movement krijgt, kan SetActive(false/true) intern gedrag resetten. Dan is deze structuur veiliger:

Bird_Root

- | - Bird_LogicRoot (eventueel altijd actief houden)
- | - Bird_VisibleRoot (door HybridObjectToggleGroup aan/uit zetten)
- | - Bird_AudioRoot (optioneel mee aan/uit zetten)

19.13 Uitbreiden naar meer objecten

Voor meer objecten gebruik je dezelfde manager en voeg je extra groups toe. Elk object krijgt een eigen index.

Index	Objectgroep	Voorbeeld
0	Bird	Bird global/local override
1	Coffee machine ambience	Lokaal of global aan/uit zetten
2	Decoratie objectgroep	Performance of comfort toggle
3	NPC / animal group	Later uitbreidbaar

Belangrijk: voeg pas nieuwe objectgroepen toe nadat de Bird versie is gebackupt.

19.14 Testplan na uitbreiding

- Test eerst in editor met 1 objectgroep.
- Controleer dat Toggle Index overall klopt.
- Test global ON/OFF.
- Test local override.
- Test global override na lokale afwijking.
- Test met twee spelers in VRChat.
- Test late join.
- Controleer dat bestaande PaperTablet toggles, tabs, VIP, screen readability en globe nog werken.

Appendix B - HybridObjectToggleManager.cs

```
using UdonSharp;
using UnityEngine;
using VRC.SDKBase;

namespace StefanieInVR
{
    [UdonBehaviourSyncMode(BehaviourSyncMode.Manual)]
    public class HybridObjectToggleManager : UdonSharpBehaviour
    {
        [Header("Default Global States")]
        [Tooltip("Index 0 = Bird. True = starts ON, False = starts OFF.")]
        public bool[] defaultGlobalStates;

        [Header("Toggle Groups")]
        [Tooltip("One group per object toggle. Example: Bird group = index 0.")]
        public HybridObjectToggleGroup[] groups;

        [Header("Debug")]
        public bool debugLogs = false;

        [UdonSynced]
        private int globalStateBits = 0;

        [UdonSynced]
        private int globalRevision = 0;

        [UdonSynced]
        private bool hasInitializedNetworkState = false;

        private int localOverrideBits = 0;
        private int localValueBits = 0;
        private int lastAppliedGlobalRevision = -1;

        private bool hasStarted = false;

        private void Start()
        {
            hasStarted = true;

            if (!hasInitializedNetworkState)
            {
                globalStateBits = BuildDefaultBits();

                if (Networking.LocalPlayer != null && Networking.IsOwner(gameObject))
                {
                    hasInitializedNetworkState = true;
                    RequestSerialization();

                    if (debugLogs)
                    {
```

```

        Debug.Log("[HybridObjectToggleManager] Initialized default global states.");
    }
}

lastAppliedGlobalRevision = globalRevision;

SendCustomEventDelayedFrames(nameof(RefreshAll), 1);
}

public override void OnDeserialization()
{
    ApplyGlobalRevisionIfNeeded();
    RefreshAll();

    if (debugLogs)
    {
        Debug.Log("[HybridObjectToggleManager] Deserialized. Revision: " + globalRevision);
    }
}

public void ToggleGlobal(int toggleIndex)
{
    if (!IsValidIndex(toggleIndex))
    {
        Debug.LogWarning("[HybridObjectToggleManager] ToggleGlobal failed. Invalid index: " + toggleIndex);
        return;
    }

    TakeOwnershipIfNeeded();

    bool currentState = GetGlobalState(toggleIndex);
    SetGlobalStateInternal(toggleIndex, !currentState);
}

public void SetGlobalOn(int toggleIndex)
{
    if (!IsValidIndex(toggleIndex))
    {
        Debug.LogWarning("[HybridObjectToggleManager] SetGlobalOn failed. Invalid index: " + toggleIndex);
        return;
    }

    TakeOwnershipIfNeeded();
    SetGlobalStateInternal(toggleIndex, true);
}

public void SetGlobalOff(int toggleIndex)
{
    if (!IsValidIndex(toggleIndex))
    {
        Debug.LogWarning("[HybridObjectToggleManager] SetGlobalOff failed. Invalid index: " + toggleIndex);
    }
}

```

```

        return;
    }

    TakeOwnershipIfNeeded();
    SetGlobalStateInternal(toggleIndex, false);
}

public void ToggleLocal(int toggleIndex)
{
    if (!IsValidIndex(toggleIndex))
    {
        Debug.LogWarning("[HybridObjectToggleManager] ToggleLocal failed. Invalid index: " + toggleIndex);
        return;
    }

    ApplyGlobalRevisionIfNeeded();

    bool currentEffectiveState = GetEffectiveState(toggleIndex);
    bool newLocalState = !currentEffectiveState;

    localOverrideBits = SetBit(localOverrideBits, toggleIndex, true);
    localValueBits = SetBit(localValueBits, toggleIndex, newLocalState);

    RefreshAll();

    if (debugLogs)
    {
        Debug.Log("[HybridObjectToggleManager] Local toggle changed. Index: " + toggleIndex + " State: " + newLocalState);
    }
}

public void ClearLocalOverride(int toggleIndex)
{
    if (!IsValidIndex(toggleIndex))
    {
        Debug.LogWarning("[HybridObjectToggleManager] ClearLocalOverride failed. Invalid index: " + toggleIndex);
        return;
    }

    localOverrideBits = SetBit(localOverrideBits, toggleIndex, false);
    RefreshAll();

    if (debugLogs)
    {
        Debug.Log("[HybridObjectToggleManager] Local override cleared. Index: " + toggleIndex);
    }
}

public void ClearAllLocalOverrides()
{
    localOverrideBits = 0;
    localValueBits = 0;

```



```

RefreshAll();

if (debugLogs)
{
    Debug.Log("[HybridObjectToggleManager] All local overrides cleared.");
}
}

public bool GetGlobalState(int toggleIndex)
{
    if (!IsValidIndex(toggleIndex))
    {
        return false;
    }

    return GetBit(globalStateBits, toggleIndex);
}

public bool GetEffectiveState(int toggleIndex)
{
    if (!IsValidIndex(toggleIndex))
    {
        return false;
    }

    ApplyGlobalRevisionIfNeeded();

    bool hasLocalOverride = GetBit(localOverrideBits, toggleIndex);

    if (hasLocalOverride)
    {
        return GetBit(localValueBits, toggleIndex);
    }

    return GetGlobalState(toggleIndex);
}

public bool HasLocalOverride(int toggleIndex)
{
    if (!IsValidIndex(toggleIndex))
    {
        return false;
    }

    ApplyGlobalRevisionIfNeeded();
    return GetBit(localOverrideBits, toggleIndex);
}

public int GetGlobalRevision()
{
    return globalRevision;
}

```

```

    }

    public void RefreshAll()
    {
        ApplyGlobalRevisionIfNeeded();

        if (groups == null)
        {
            return;
        }

        for (int i = 0; i < groups.Length; i++)
        {
            if (groups[i] != null)
            {
                groups[i].RefreshFromManager();
            }
        }
    }

    private void SetGlobalStateInternal(int toggleIndex, bool newState)
    {
        globalStateBits = SetBit(globalStateBits, toggleIndex, newState);

        globalRevision++;

        hasInitializedNetworkState = true;

        localOverrideBits = 0;
        localValueBits = 0;
        lastAppliedGlobalRevision = globalRevision;

        RequestSerialization();
        RefreshAll();

        if (debugLogs)
        {
            Debug.Log("[HybridObjectToggleManager] Global state changed. Index: " + toggleIndex + " State: " + newState + "
Revision: " + globalRevision);
        }
    }

    private void ApplyGlobalRevisionIfNeeded()
    {
        if (lastAppliedGlobalRevision != globalRevision)
        {
            localOverrideBits = 0;
            localValueBits = 0;
            lastAppliedGlobalRevision = globalRevision;

            if (debugLogs && hasStarted)
            {

```

```

        Debug.Log("[HybridObjectToggleManager] Global revision changed. Local overrides cleared.");
    }
}

```

```

private void TakeOwnershipIfNeeded()
{
    if (Networking.LocalPlayer == null)
    {
        return;
    }

    if (!Networking.IsOwner(gameObject))
    {
        Networking.SetOwner(Networking.LocalPlayer, gameObject);
    }
}

```

```

private int BuildDefaultBits()
{
    int bits = 0;

    if (defaultGlobalStates == null)
    {
        return bits;
    }

    int max = defaultGlobalStates.Length;

    if (max > 31)
    {
        max = 31;
    }

    for (int i = 0; i < max; i++)
    {
        bits = SetBit(bits, i, defaultGlobalStates[i]);
    }

    return bits;
}

```

```

private bool IsValidIndex(int toggleIndex)
{
    if (toggleIndex < 0)
    {
        return false;
    }

    if (toggleIndex > 30)
    {
        return false;
    }
}

```

```

    }

    return true;
}

private bool GetBit(int bits, int index)
{
    int mask = 1 << index;
    return (bits & mask) != 0;
}

private int SetBit(int bits, int index, bool value)
{
    int mask = 1 << index;

    if (value)
    {
        return bits | mask;
    }

    return bits & ~mask;
}
}
}

```

Appendix C - HybridObjectToggleInput.cs

```
using UdonSharp;
using UnityEngine;

namespace StefanieInVR
{
    [UdonBehaviourSyncMode(BehaviourSyncMode.None)]
    public class HybridObjectToggleInput : UdonSharpBehaviour
    {
        [Header("Required")]
        public HybridObjectToggleManager manager;

        [Tooltip("Index of the object group. Bird = 0.")]
        public int toggleIndex = 0;

        [Header("Button Type")]
        [Tooltip("ON = this button changes the synced global state. OFF = this button changes only the local player view.")]
        public bool isGlobalButton = false;

        [Header("Optional Fixed Action")]
        [Tooltip("If true, this button always sets ON instead of toggling.")]
        public bool forceOn = false;

        [Tooltip("If true, this button always sets OFF instead of toggling.")]
        public bool forceOff = false;

        [Header("Local Only Option")]
        [Tooltip("Only for local buttons. If true, this clears the local override and returns to the global state.")]
        public bool clearLocalOverrideInstead = false;

        [Header("Debug")]
        public bool debugLogs = false;

        public override void Interact()
        {
            Press();
        }

        public void Press()
        {
            if (manager == null)
            {
                Debug.LogWarning("[HybridObjectToggleInput] No manager assigned.");
                return;
            }

            if (toggleIndex < 0 || toggleIndex > 30)
            {
                Debug.LogWarning("[HybridObjectToggleInput] Invalid toggleIndex: " + toggleIndex);
                return;
            }
        }
    }
}
```

```

    if (isGlobalButton)
    {
        PressGlobal();
    }
    else
    {
        PressLocal();
    }
}

public void PressGlobal()
{
    if (manager == null)
    {
        Debug.LogWarning("[HybridObjectToggleInput] PressGlobal failed. No manager assigned.");
        return;
    }

    if (forceOn)
    {
        manager.SetGlobalOn(toggleIndex);
    }
    else if (forceOff)
    {
        manager.SetGlobalOff(toggleIndex);
    }
    else
    {
        manager.ToggleGlobal(toggleIndex);
    }

    if (debugLogs)
    {
        Debug.Log("[HybridObjectToggleInput] Global button pressed. Index: " + toggleIndex);
    }
}

public void PressLocal()
{
    if (manager == null)
    {
        Debug.LogWarning("[HybridObjectToggleInput] PressLocal failed. No manager assigned.");
        return;
    }

    if (clearLocalOverrideInstead)
    {
        manager.ClearLocalOverride(toggleIndex);
    }
    else
    {

```

```
        manager.ToggleLocal(toggleIndex);
    }

    if (debugLogs)
    {
        Debug.Log("[HybridObjectToggleInput] Local button pressed. Index: " + toggleIndex);
    }
}
}
```

Appendix D - HybridObjectToggleGroup.cs

```
using UdonSharp;
using UnityEngine;
using UnityEngine.UI;

namespace StefanieInVR
{
    [UdonBehaviourSyncMode(BehaviourSyncMode.None)]
    public class HybridObjectToggleGroup : UdonSharpBehaviour
    {
        [Header("Required")]
        public HybridObjectToggleManager manager;

        [Tooltip("Index of this object group. Bird = 0.")]
        public int toggleIndex = 0;

        [Header("Controlled Objects")]
        [Tooltip("Objects that are active when the final/effective state is ON.")]
        public GameObject[] objectsOnWhenEnabled;

        [Tooltip("Objects that are active when the final/effective state is OFF. Usually leave empty.")]
        public GameObject[] objectsOnWhenDisabled;

        [Header("Global Toggle Sprite")]
        [Tooltip("Image on the GLOBAL button that should swap between ON and OFF sprite.")]
        public Image globalToggleImage;

        [Header("Local Toggle Sprite")]
        [Tooltip("Image on the LOCAL button that should swap between ON and OFF sprite.")]
        public Image localToggleImage;

        [Header("Sprites")]
        [Tooltip("Sprite shown when the state is ON.")]
        public Sprite onSprite;

        [Tooltip("Sprite shown when the state is OFF.")]
        public Sprite offSprite;

        [Header("Optional Local Override Indicator")]
        [Tooltip("Optional object that becomes active when this player has a local override.")]
        public GameObject localOverrideIndicator;

        [Header("Options")]
        public bool refreshOnStart = true;

        [Header("Debug")]
        public bool debugLogs = false;

        private void Start()
        {
            if (refreshOnStart)
```



```

    {
        SendCustomEventDelayedFrames(nameof(RefreshFromManager), 1);
    }
}

public void RefreshFromManager()
{
    if (manager == null)
    {
        Debug.LogWarning("[HybridObjectToggleGroup] No manager assigned.");
        return;
    }

    bool globalState = manager.GetGlobalState(toggleIndex);
    bool effectiveState = manager.GetEffectiveState(toggleIndex);
    bool hasLocalOverride = manager.HasLocalOverride(toggleIndex);

    ApplyControlledObjects(effectiveState);
    ApplyGlobalSprite(globalState);
    ApplyLocalSprite(effectiveState);
    ApplyLocalOverrideIndicator(hasLocalOverride);

    if (debugLogs)
    {
        Debug.Log("[HybridObjectToggleGroup] Refreshed index: " + toggleIndex + " Global: " + globalState + " Effective: " +
effectiveState + " Local override: " + hasLocalOverride);
    }
}

private void ApplyControlledObjects(bool effectiveState)
{
    SetGameObjects(objectsOnWhenEnabled, effectiveState);
    SetGameObjects(objectsOnWhenDisabled, !effectiveState);
}

private void ApplyGlobalSprite(bool globalState)
{
    if (globalToggleImage == null)
    {
        return;
    }

    Sprite targetSprite = globalState ? onSprite : offSprite;

    if (targetSprite == null)
    {
        return;
    }

    if (globalToggleImage.sprite != targetSprite)
    {
        globalToggleImage.sprite = targetSprite;
    }
}

```

```

    }
}

private void ApplyLocalSprite(bool effectiveState)
{
    if (localToggleImage == null)
    {
        return;
    }

    Sprite targetSprite = effectiveState ? onSprite : offSprite;

    if (targetSprite == null)
    {
        return;
    }

    if (localToggleImage.sprite != targetSprite)
    {
        localToggleImage.sprite = targetSprite;
    }
}

private void ApplyLocalOverrideIndicator(bool hasLocalOverride)
{
    if (localOverrideIndicator == null)
    {
        return;
    }

    if (localOverrideIndicator.activeSelf != hasLocalOverride)
    {
        localOverrideIndicator.SetActive(hasLocalOverride);
    }
}

private void SetGameObjects(GameObject[] objects, bool state)
{
    if (objects == null)
    {
        return;
    }

    for (int i = 0; i < objects.Length; i++)
    {
        if (objects[i] != null)
        {
            if (objects[i].activeSelf != state)
            {
                objects[i].SetActive(state);
            }
        }
    }
}

```

```
}  
}  
}  
}
```